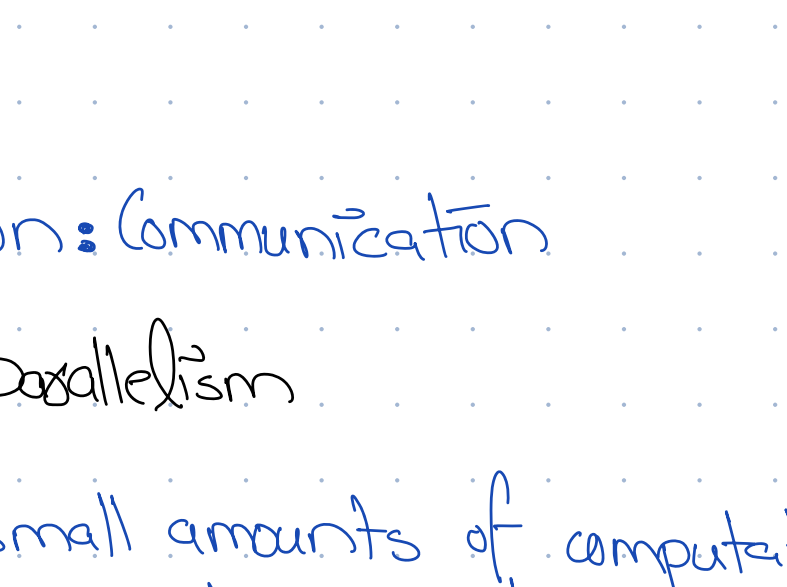
Load Balancing:

- Practice of distributing approximately equal amounts of work among threads.
  - All threads are kept busy all the time
  - It can be considered a minization of thread idle time.
- If all tasks are subject to barrier synchronization point, the slowest task will determine the overall performance.
- How to achieve load balancing?
  - Equally partition the work each task recieves
  - Use dynamic work assignment.

Granularity:

- Computation : Communication

1. Fine grained parallelism
   - Relatively small amounts of computational work done between communication event.
     - Low computation to communication ratio & less opportunity for performance enhancement
     - Implies high communication overhead



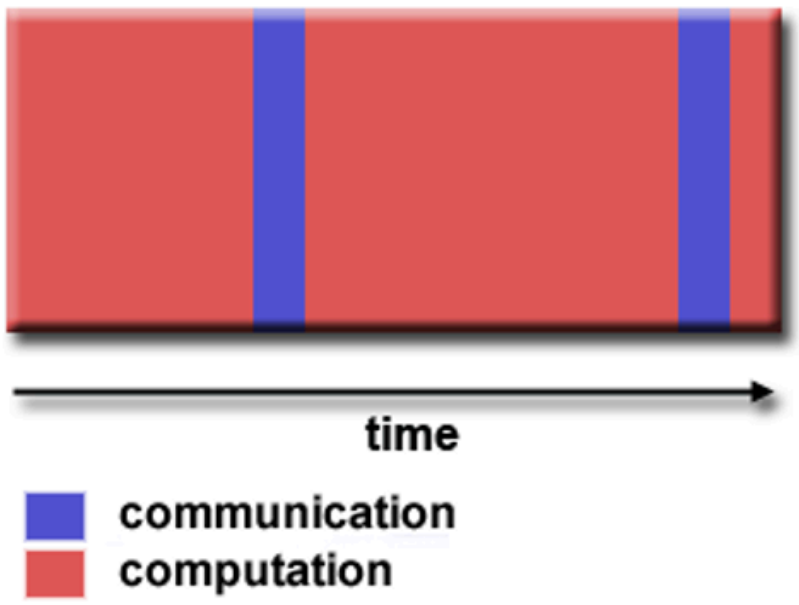
2. Coarse grained Parallelism

↳ Relatively large amounts of computational work done between communication event.

↳ High computation to communication ratio

↳ Implies more opportunity for performance increase

↳ Harder to load balance



GPU is best suited for fine grained parallelism

Message Passing interface is best suited for coarse grained parallelism

Input & Output

↳ I/O are bad to parallelism

↳ I/O require alot of time.

↳ Parallel I/O systems may be not available:

- Write operation can result in file overwriting
- Read operation depend on file server's ability to handle multiple read requests.

↳ process 0, in distributed memory programs, and thread 0 in shared memory programs will access stdin

Whereas, all processes/threads access stdout & stderr in both

- Only a single process/thread will attempt to access any single file.

- Because of the indeterminacy of orders to output to stdout,
  - Often, a single process/thread will be used for all output to stdout other than debugging

Why the Slide Says "All Can Access `stdout` / `stderr`":

- In **distributed memory** programs, each process has its own local memory, and can **access its own stdout** or stderr independently.
  - In **shared memory** programs, each thread can also access `stdout` / `stderr`, since all threads share the same memory space.
- But **only one process/thread** should **write** to `stdout` (to avoid mixing the output from multiple processes), though **any process or thread** could technically print debug information to **stderr**.

Scalability:

Strongly Scalable:

Increase the number of processes/threads without increasing problem size.

- Total problem size stays fixed as more processors are added
  - Goal is to run the same problem size faster.

Weakly Scalable:

keep efficiency fixed by increasing the problem size at same rate as we increase no. of processors used.

- Total problem size is proportional to number of processors
- Goal is to run larger problem in same amount of time.

Data Dependencies:

- A dependence exists b/w program statements when the order of statement execution affects the results of the problem.
- A data dependence results from multiple use of the same location in storage by different tasks.

How to handle?

For distributed memory architectures, communicate required data at synchronization points

For shared memory architectures, synchronize read/write operations between tasks

Synchronization Background

Cooperating Process:

- A cooperating process is a process that interacts with or is influenced by other processes. In other words, it depends on or affects the behavior of other processes in the system i.e they work on the same data.
- **These processes may:**
  - **Share the same memory:** This means they can access both code and data in a shared space.
  - **Share only specific data:** In this case, they use things like shared memory or message passing to communicate and exchange data.

Process Interruptions:

- A process can be interrupted at any point while it is running. This means that, for example, the CPU could be taken away from a process in the middle of its execution and assigned to another process to run.
- After that, when the original process resumes, it might not know that it was interrupted, which can lead to issues with the data it's using.

Concurrent Access to Shared Data:

- When multiple processes are running and they are sharing data, there is a risk of data inconsistency.
- For example, one process might change some data while another process is reading it. This could lead to one process using outdated or incorrect data, causing errors or bugs in the system.
- This happens because different processes may access and modify the same data at the same time, and since processes can be interrupted at any moment, the data can get corrupted or inconsistent.